

LangChain — Prompts & Structured Output

- Input and Output in LLM

1. What is a Prompt?

- A prompt is the instruction or input that we give to an AI model to tell it what we want.

2. Types of Prompts

- we can divide prompts into two basic types:
 - 1) Static Prompt
 - 2) Dynamic Prompt

1) What is a Static Prompt?

- A static prompt is a fixed prompt whose content does not change during execution.
- Ex : prompt = "Explain Python in simple language."

```
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
    model="gemini-3.8-flash",
    google_api_key="AQ.Ab8RN6LRgG4bmjJSBppac6KvJWz5xJBVoFj_7LQK1cWcU9ecPw"
)

res = model.invoke("who is pm of india")

print(res.text)
```

2) What is a Dynamic Prompt?

- A dynamic prompt changes according to the user's input.
- For example, the user can enter a topic.

```
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
```

```

    model="gemini-3.8-flash",
    google_api_key=""
)

topic = input("Enter your topic: ")

prompt = f"Explain {topic} in simple language with an example."

res = model.invoke(prompt)

print(res.text)

```

3. PromptTemplate

- In **LangChain**, a `PromptTemplate` is a way to create a **reusable prompt with dynamic values**.
- Instead of writing a new prompt every time, you create a template with **placeholders** like `{country}`, `{name}`, or `{topic}`.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import PromptTemplate

model = ChatGoogleGenerativeAI(
    model="gemini-3.8-flash",
    google_api_key="AQ.Ab8RN6LRgG4bmjJSBppac6KvJWz5xJBVoFj_7LQK1cWcU9ecPw"
)

country = input("Enter Country :")

prompt = PromptTemplate(
    template="who is pm of {country}",
    input_variables=["country"]
)

complete_prompt = prompt.invoke({"country":country})

ans = model.invoke(complete_prompt)

print(ans.text)

```

detailed dynamic prompt:

```
"""
Act as an expert and highly experienced {topic} teacher and explain {topic} to me
as a beginner using simple, clear, and practical language.
Start from the basics, explain the important concepts step-by-step, and use real-
world examples and simple code examples wherever applicable.
Explain not only what {topic} is, but also why it is used, how it works, where it
is used in real-world projects, and how it is different from related concepts.
For every important concept, mention common mistakes beginners make and give
practical tips that will help me understand and remember it.
At the end, give me a small practical task or mini-project so I can practice what
I learned and check my understanding.
Finally, prepare 5 beginner and 5 intermediate interview questions with clear
answers, followed by the most important points I should remember for an
interview.
"""
```

ChatBot Example :

```
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    google_api_key="AQ.Ab8RN6LRgG4bmjJSBppac6KvJWz5xJBVoFj_7LQK1cWcU9ecPw"
)

condition = True

while condition==True:

    question = input("Enter Your Question :")
    if question=="exit":
        break
    else:
        ans = model.invoke(question)
        print(ans.text)
```

4. Structured Output

- Structured output means instructing an AI model to return its response in a fixed, predefined format or structure, instead of normal free-form text.
- Python is a programming language. It is easy to learn...

```
{  
  "topic": "Python",  
  "difficulty": "Beginner",  
  "use_cases": ["Web Development", "AI", "Data Science"]  
}
```

5. Why do we need structured output?

- Normal AI output is easy for humans to understand, but another software/system may not understand it properly because the response can be in different formats every time.
- When we want to integrate AI output with another system, such as a database, API, website, or application, we need the data in a specific predefined structure.
- Therefore, we use Structured Output to make the AI return data in the required format, such as JSON, dictionary, or a defined schema, so that another system can easily read and process it.

In simple words:

- Human → Normal text is enough.
 - System → Data must follow a proper structure.
- That's why we use Structured Output.

6. Where is structured output used?

- 1) AI Chatbots
- 2) Resume parsing

- 3) E-commerce
- 4) Data extraction
- 5) APIs and backend applications

7. ways to get Structured Output

- 1) TypedDict
- 2) Pydantic
- 3) JSON Schema
- 4) Dataclasses
- 5) JSON / dictionary format

1) TypedDict

- TypedDict is a Python feature used to define the expected structure of a dictionary by specifying the field names and their data types.

```
from typing import TypedDict

class person(TypedDict):
    name:str
    age:int
    contact:int

info:person={
    "name": "Rahul",
    "age": "twenty",
    "number": "9876543210"
}

print(info)
```

2) Pydantic

- Pydantic is a Python library used to define the structure and data types of data and automatically validate the data according to that structure.

```
from pydantic import BaseModel
```

```

class Person(BaseModel):
    name: str
    age: int
    contact: str

info = Person(
    name="Rahul",
    age="44",
    contact="9876543210"
)

print(info)

```

Example :

```

from langchain_google_genai import ChatGoogleGenerativeAI
from pydantic import BaseModel , Field
from typing import Annotated

model = ChatGoogleGenerativeAI(
    model="gemini-3.8-flash",
    google_api_key = "AQ.Ab8RN6LRgG4bmjJSBppac6KvJWz5xJBVoFj_7LQK1cWcU9ecPw"
)

class outputStuct(BaseModel):
    name:Annotated[str , Field(description="this is for users name")]
    age:Annotated[int , Field(description="this is for users name")]
    contact:Annotated[int , Field(description="this is for users name")]

output_format = model.with_structured_output(outputStuct)

result = output_format.invoke("my name is demouser and my age is 25 and my
contact number is 8989677654")

print(result)

```

8. What is Output Parsing?

- Output parsing is the process of taking the raw output generated by an LLM and converting it into a specific, structured format that your application can easily use.
- Output Parser converts the LLM's response into this required format.

9. Why Do We Use an Output Parser?

- Some LLMs support structured output directly, so we can give them a schema and get data in that format.
- But some LLMs only return normal text. In that case, we use an Output Parser to convert the normal text into the structure our program needs.

Example :

```
from langchain_huggingface import HuggingFaceEndpoint , ChatHuggingFace
from pydantic import BaseModel , Field
from typing import Annotated
from langchain_core.output_parsers import PydanticOutputParser

model = HuggingFaceEndpoint(
    repo_id="meta-llama/Llama-3.1-8B-Instruct",
    task="text-generation",
    huggingfacehub_api_token="hf_SgOuzfRPZhdJuuhdrYjbeajSmKoEvSLjj0"
)

class persontStuct(BaseModel):
    name: Annotated[str, Field(description="give user name here")]
    age: Annotated[int, Field(description="give user age here")]
    contact: Annotated[int, Field(description="give user contact here")]

parser = PydanticOutputParser(pydantic_object=persontStuct)

llm_model = ChatHuggingFace(llm=model)

prompt = f"my name is demouser my age is 22 my contact number is 8989898989
{parser.get_format_instructions()}"

result = llm_model.invoke(prompt)
print(result.content)
```